



Facultad de
**Ciencias Sociales y
Tecnologías de la Información**
Talavera de la Reina. UCLM

MEMORIA

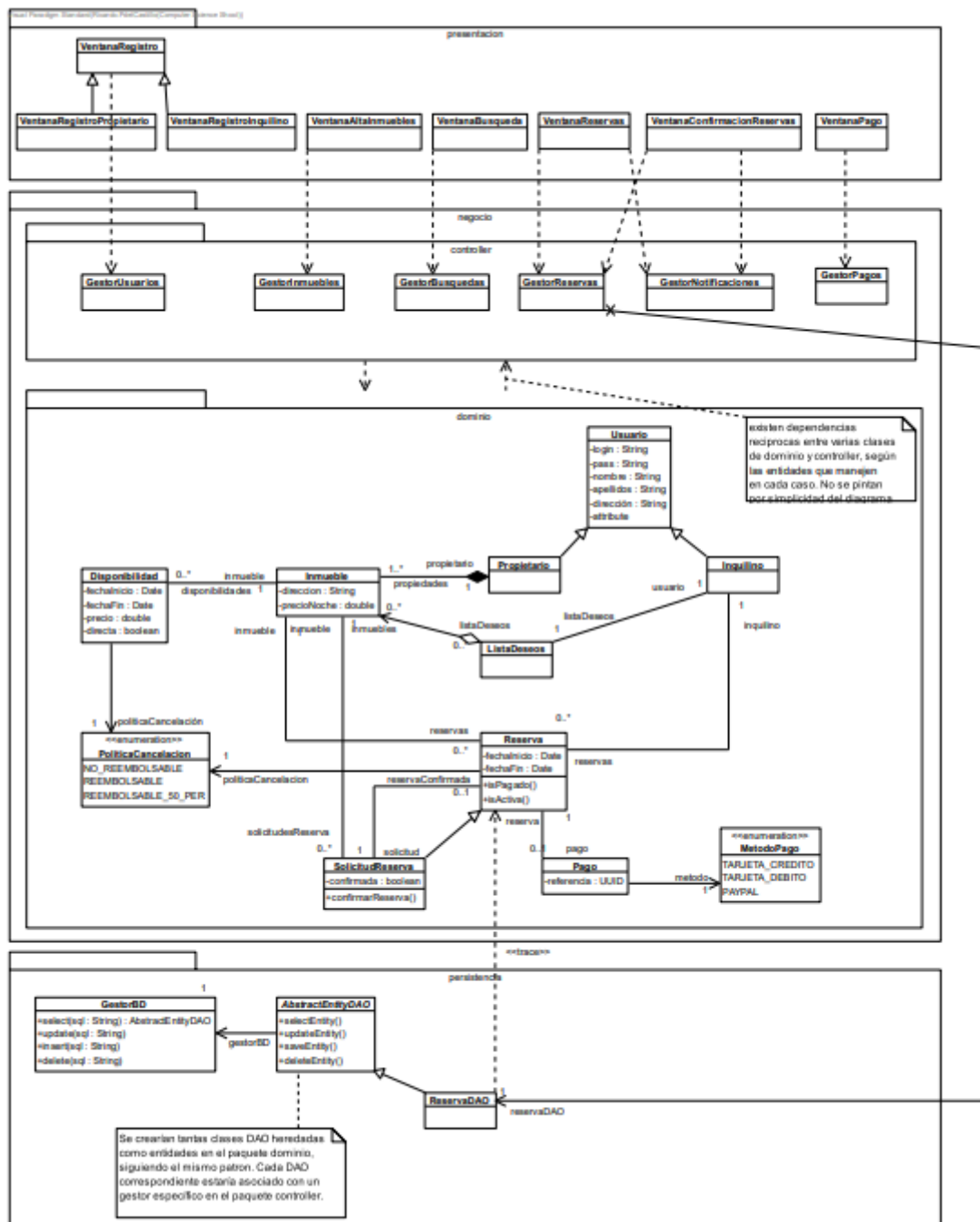
CLAUDIA MARTÍN – CRISTHIAN GARCÍA – PEDRO MARTÍN
INGENIERIA DEL SOFTWARE II

PROYECTO ISO II

CONTENIDO

1.	Introducción	2
2.	Planificación y gestión	3
2.1.	Metodología de desarrollo (Scrum)	3
2.2.	Roles del equipo	4
2.3.	Herramientas de planificación	4
2.4.	Planificación por Sprints	4
2.5.	Distribución de tareas por integrante	10
3.	Codificación	10
3.1.	Tecnologías y arquitectura	10
3.2.	Componentes principales	10
4.	Gestión de configuración	11
4.1.	Control de versiones	11
4.2.	Gestión de dependencias y compilación	11
4.3.	Integración y despliegue	11
5.	Sonar (Análisis inicial)	12
5.1.	Resultados del análisis inicial	12
5.2.	Resultado segundo análisis	12
5.3.	Propuesta de configuración alternativa	13
5.4.	Issues seleccionadas y explicación	14
5.5.	Resultado Ultimo análisis	17
5.6.	Conclusiones del análisis	17
5.7.	Pruebas unitarias (JUnit)	18
5.7.1.	Introducción a las pruebas unitarias	18
5.7.2.	Tests implementados (7 explicados)	18
5.8.	Informes de pruebas y cobertura (Surefire / JaCoCo)	26
5.9.	Mantenimiento del software	27
5.9.1.	Mantenimiento Correctivo	27
5.9.2.	Mantenimiento Perfectivo	28
5.9.3.	Mantenimiento Preventivo	28
5.10.	Comparativa de métricas	28
6.	Conclusión	28

PROYECTO ISO II



2. PLANIFICACIÓN Y GESTIÓN

2.1. METODOLOGÍA DE DESARROLLO (SCRUM)

Para el desarrollo del proyecto se utilizó una metodología ágil basada en Scrum y Kanban.

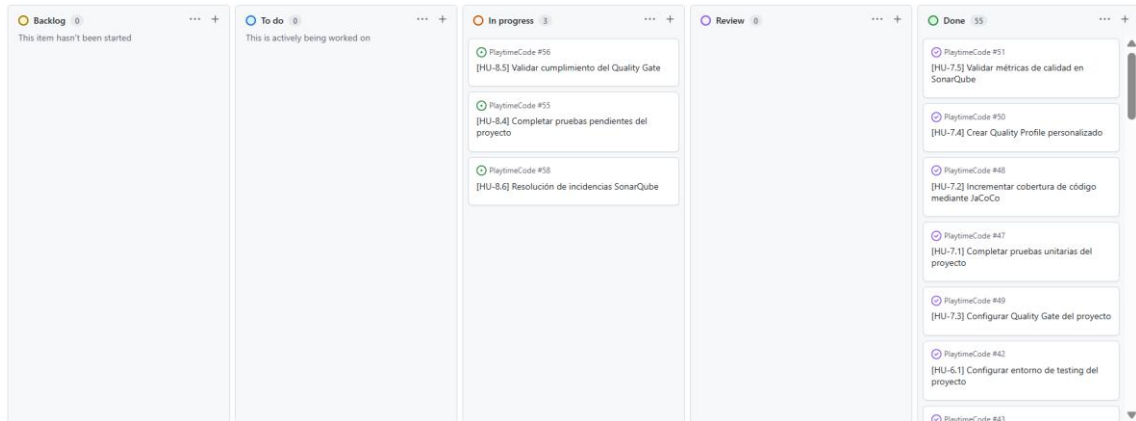
Las fases de Scrum aplicadas fueron:

1. Sprint Planning: selección de historias de usuario y definición de tareas.
2. Daily Scrum: coordinación diaria del equipo.
3. Sprint Review: evaluación de resultados parciales.

PROYECTO ISO II

4. Retrospective: análisis de eficiencia y mejoras de proceso.

Cada sprint tenía objetivos concretos y funcionalidades específicas.



2.2. ROLES DEL EQUIPO

El equipo cuenta con tres integrantes, distribuidos en los siguientes roles:

Rol	Responsabilidad principal
Product Owner	Definir y priorizar el backlog, validar entregas
Scrum Master	Asegurar la correcta aplicación de Scrum y eliminar bloqueos
Desarrollador Backend	Implementar la lógica del sistema, entidades y persistencia
Desarrollador Frontend	Desarrollar vistas, controladores y diseño web
Responsable de Testing y Calidad	Implementar pruebas unitarias, medir cobertura y supervisar la calidad del código mediante SonarQube

Los roles no fueron estrictamente exclusivos, permitiendo la colaboración conjunta en tareas de desarrollo, testing y mantenimiento del sistema.

2.3. HERRAMIENTAS DE PLANIFICACIÓN

- GitHub Projects: gestión del backlog y tareas mediante tablero Kanban (To Do, In Progress, Done)
- GitHub Milestones: control de hitos y seguimiento por sprints
- Google Docs: documentación
- Maven: soporte de gestión de dependencias y ejecución

2.4. PLANIFICACIÓN POR SPRINTS

El trabajo se estructuró en siete Sprints principales, registrados como milestones en GitHub:

Sprint 0 – Setup (1 semana)

PROYECTO ISO II

El Sprint 0 estuvo enfocado en la preparación del entorno de desarrollo y en la definición de la base inicial del proyecto PlaytimeCode. Durante esta fase se configuraron las herramientas necesarias y se estableció la estructura principal del sistema para facilitar el desarrollo posterior.

En este sprint se creó el repositorio del proyecto en GitHub, se configuró el entorno de desarrollo con Eclipse y Maven y se preparó un proyecto Spring Boot siguiendo una arquitectura MVC.

Las tareas realizadas durante este sprint incluyeron:

- Creación del repositorio en GitHub
- Configuración de Git y ramas del proyecto
- Preparación del entorno en Eclipse
- Configuración del proyecto Maven y Spring Boot
- Definición de la estructura MVC
- Organización inicial del backlog y milestones

Este sprint permitió disponer de una base sólida y organizada sobre la que desarrollar el resto de las funcionalidades del proyecto.

Sprint 1 – Usuarios y Autenticación (1 semana)

El Sprint 1 estuvo centrado en la implementación del sistema de usuarios y autenticación de la aplicación. Durante esta fase se desarrollaron las funcionalidades necesarias para permitir el registro e inicio de sesión de los usuarios, así como la gestión de roles dentro del sistema.

Se implementaron dos tipos principales de usuario: propietario e inquilino, permitiendo adaptar las funcionalidades visibles según el rol del usuario autenticado.

Las tareas realizadas durante este sprint incluyeron:

- Implementación del registro de usuarios
- Desarrollo del sistema de inicio de sesión
- Gestión de roles de usuario
- Validación de datos de entrada
- Control de errores en autenticación
- Desarrollo de formularios de login y registro
- Implementación de lógica backend de autenticación
- Validación y revisión de pruebas funcionales

Este sprint permitió disponer de un sistema básico de autenticación y gestión de usuarios sobre el que desarrollar el resto de las funcionalidades del proyecto.

Sprint 2 – Gestión de propiedades (1 semana)

El Sprint 2 estuvo centrado en el desarrollo del sistema de gestión de propiedades para los usuarios con rol propietario. Durante esta fase se implementaron las funcionalidades necesarias para permitir la creación, edición, eliminación y visualización de inmuebles dentro de la plataforma.

Además, se incorporó un control de acceso para asegurar que únicamente los usuarios con rol propietario pudieran acceder a las funcionalidades de gestión de propiedades.

PROYECTO ISO II

Las tareas realizadas durante este sprint incluyeron:

- Implementación del sistema de creación de propiedades
- Desarrollo de edición y eliminación de inmuebles
- Listado de propiedades asociadas al propietario
- Control de acceso según roles de usuario
- Desarrollo de formularios de gestión de propiedades
- Implementación de lógica CRUD en backend
- Validaciones de datos y control de errores
- Revisión y validación mediante Pull Requests

El desarrollo se organizó utilizando ramas feature independientes para las operaciones de creación, edición, eliminación y listado de propiedades, integrándose posteriormente en la rama develop mediante Pull Requests.

Este sprint permitió incorporar la gestión completa de inmuebles y establecer una de las funcionalidades principales del sistema.

Sprint 3 – Gestión de inquilinos + filtros (1 semana)

El Sprint 3 estuvo centrado en el desarrollo de las funcionalidades orientadas a los usuarios con rol inquilino, permitiendo la visualización de propiedades disponibles y la búsqueda de inmuebles mediante filtros personalizados según las preferencias del usuario.

Durante este sprint se implementó la interfaz principal del inquilino y se desarrolló un sistema de búsqueda que permite consultar propiedades aplicando distintos criterios, como precio, ubicación o características del inmueble.

Las tareas realizadas durante este sprint incluyeron:

- Desarrollo de la interfaz principal del inquilino
- Visualización de propiedades disponibles
- Implementación del sistema de búsqueda de inmuebles
- Aplicación de filtros personalizados sobre propiedades
- Integración frontend-backend para consulta de datos
- Validación de filtros y resultados obtenidos
- Manejo de errores y control de búsquedas vacías
- Pruebas funcionales y revisión mediante Pull Requests

El desarrollo se organizó mediante funcionalidades agrupadas relacionadas con la gestión de inquilinos y filtros, integrándose progresivamente en la rama develop mediante Pull Requests.

Este sprint permitió mejorar significativamente la experiencia del usuario, facilitando la búsqueda y selección de propiedades según distintos criterios definidos por el inquilino.

Sprint 4 – Filtros y estabilización del sistema (1 semana)

El Sprint 4 estuvo centrado en completar la funcionalidad de búsqueda y filtros de propiedades para los inquilinos, así como en continuar con la estabilización general del sistema y la corrección de errores detectados durante el desarrollo en la rama develop.

PROYECTO ISO II

Debido a diversos problemas técnicos surgidos en la integración de funcionalidades anteriores, parte del trabajo inicialmente previsto para el Sprint 3 tuvo que trasladarse a este sprint, dedicándose tiempo adicional a la revisión y estabilización del sistema.

Las tareas realizadas durante este sprint incluyeron:

- Finalización del sistema de filtros de propiedades
- Integración completa entre frontend y backend
- Corrección de errores detectados en develop
- Validación del funcionamiento general del sistema
- Revisión de búsquedas y resultados obtenidos
- Mejora de estabilidad y control de errores
- Integración progresiva mediante Pull Requests hacia develop

El desarrollo continuó sobre ramas feature específicas, integrando las funcionalidades una vez validadas correctamente.

Este sprint permitió completar el sistema de búsqueda de propiedades y mejorar considerablemente la estabilidad general de la aplicación antes de continuar con nuevas funcionalidades.

Sprint 5 – Lista de deseos y política de cancelación (1 semana)

El Sprint 5 estuvo centrado en la implementación de funcionalidades avanzadas orientadas a los usuarios con rol inquilino. Durante esta fase se desarrolló un sistema de lista de deseos, la funcionalidad de alquiler de inmuebles y la visualización de políticas de cancelación antes de confirmar una reserva.

Se implementó la posibilidad de guardar inmuebles favoritos para facilitar el acceso rápido a propiedades de interés, así como la gestión del alquiler y la actualización automática de disponibilidad de los inmuebles tras realizar una reserva.

Las tareas realizadas durante este sprint incluyeron:

- Implementación de la lista de deseos del usuario
- Añadir y eliminar inmuebles favoritos
- Visualización de inmuebles guardados
- Implementación de la funcionalidad de alquiler de inmuebles
- Registro de alquileres en el sistema
- Actualización de disponibilidad de propiedades
- Visualización de políticas de cancelación
- Integración frontend-backend para reservas y wishlist
- Validaciones funcionales y revisión mediante Pull Requests

El desarrollo se organizó mediante funcionalidades independientes integradas progresivamente en la rama develop mediante Pull Requests.

Este sprint permitió ampliar considerablemente las funcionalidades disponibles para el inquilino, mejorando la experiencia de usuario y añadiendo opciones más avanzadas de gestión de reservas y favoritos.

PROYECTO ISO II

Sprint 6 – Testing y calidad del proyecto (1 semana)

El Sprint 6 estuvo centrado en la mejora de la calidad y mantenibilidad del proyecto mediante la implementación de pruebas automáticas y la integración de herramientas de análisis estático de código.

Durante este sprint se configuró el entorno de testing y se desarrollaron pruebas unitarias sobre distintos controladores del sistema utilizando JUnit. Además, se incorporó JaCoCo para medir la cobertura del código y SonarQube para analizar la calidad general del proyecto y detectar posibles problemas de mantenibilidad, duplicación o cobertura insuficiente.

Las tareas realizadas durante este sprint incluyeron:

- Configuración del entorno de testing
- Implementación de tests unitarios sobre controladores
- Validación de respuestas HTTP y comportamiento esperado
- Ejecución automática de pruebas mediante Maven
- Configuración de JaCoCo para medición de cobertura
- Generación de informes de cobertura del proyecto
- Integración de cobertura con SonarQube
- Revisión de calidad del código y detección de problemas
- Identificación de código no cubierto por tests
- Validación de cobertura y resultados obtenidos

El trabajo se organizó siguiendo una estrategia orientada a calidad continua, utilizando pruebas automáticas e integración con herramientas de análisis para mejorar progresivamente la estabilidad y mantenibilidad del sistema.

Este sprint permitió aumentar significativamente la calidad del proyecto, verificar el correcto funcionamiento de distintas funcionalidades y disponer de métricas reales de cobertura y calidad del código.

Sprint 7 – Finalización del Testing y Definición de Estándares de Calidad (1 semana)

El Sprint 7 estuvo centrado en finalizar la fase de testing del proyecto y en establecer estándares de calidad que permitan mantener un control continuo sobre el estado del sistema durante el desarrollo. Para ello, se amplió la cobertura de pruebas automáticas y se configuraron mecanismos de evaluación de calidad mediante SonarQube.

Durante este sprint se trabajó tanto en la ampliación de tests como en la definición de métricas y reglas de calidad mediante Quality Gates y Quality Profiles, permitiendo evaluar automáticamente el estado del proyecto en términos de cobertura, mantenibilidad y fiabilidad.

Las tareas realizadas durante este sprint incluyeron:

- Finalización y ampliación de los tests del sistema
- Incremento de la cobertura de código del proyecto
- Validación de funcionalidades principales y críticas
- Ejecución automática de pruebas mediante Maven
- Medición de cobertura utilizando JaCoCo
- Integración de métricas de cobertura con SonarQube

PROYECTO ISO II

- Configuración de Quality Gates en SonarQube
- Creación y ajuste de Quality Profiles personalizados
- Aplicación de reglas de análisis estático de código
- Seguimiento de métricas y calidad del proyecto

El trabajo se organizó diferenciando tareas de backend, calidad y QA, permitiendo mejorar progresivamente la cobertura y estabilidad del sistema mientras se definían estándares de calidad automáticos para futuras iteraciones.

Este sprint permitió consolidar la calidad del proyecto, establecer controles automáticos de validación y disponer de métricas reales sobre cobertura, mantenibilidad y calidad del código.

Sprint 8 – Mantenimiento correctivo y validación de calidad final

El Sprint 8 estuvo centrado en la realización de tareas de mantenimiento correctivo y preventivo sobre el proyecto, con el objetivo de mejorar la calidad general del software antes de su entrega final. Durante esta fase se trabajó sobre las incidencias detectadas por SonarQube en sprints anteriores, corrigiendo bugs, eliminando code smells y refactorizando el código para mejorar su mantenibilidad y legibilidad.

Además, se completaron las pruebas pendientes del proyecto, se corrigieron los tests fallidos derivados de cambios en la firma de métodos de los controladores y se validaron las métricas finales de cobertura y calidad mediante JaCoCo y SonarQube.

Las tareas realizadas durante este sprint incluyeron:

Corrección de code smells detectados por SonarQube

Resolución de bugs identificados en el análisis estático

Refactorización de clases y métodos para reducir la complejidad cognitiva

Extracción de literales duplicados a constantes

Añadir comentarios justificativos en constructores vacíos requeridos por JPA

Añadir assertions a tests sin validaciones explícitas

Añadir llamadas a `setComplete()` en controladores con `@SessionAttributes` que posteriormente desechamos por errores

Corrección de tests fallidos por cambios en la firma de métodos de controladores

Ejecución completa de la suite de tests del proyecto

Verificación de cobertura final con JaCoCo

Revisión y cumplimiento del Quality Gate definido en SonarQube

Revisión final del Quality Profile personalizado

Reducción de la deuda técnica acumulada durante el desarrollo

PROYECTO ISO II

El trabajo se organizó sobre la rama mantenimiento, creada específicamente para gestionar todos los cambios de forma aislada, integrándose posteriormente en la rama develop mediante un Pull Request una vez validada la calidad del proyecto.

Este sprint permitió cerrar el ciclo de desarrollo con un proyecto estable, con incidencias de calidad resueltas y métricas de cobertura y mantenibilidad validadas, dejando el sistema preparado para su entrega final.

2.5. DISTRIBUCIÓN DE TAREAS POR INTEGRANTE

Integrante	Roles asignados
Cristhian García	Product Owner, Scrum Master, Desarrollador Backend
Claudia Martín	Desarrolladora Backend, Responsable de Testing y Calidad, Apoyo Frontend
Pedro Martín	Desarrollador Backend, Apoyo Frontend, Responsable de Testing y Calidad

3. CODIFICACIÓN

3.1. TECNOLOGÍAS Y ARQUITECTURA

El proyecto se desarrolló utilizando Spring Boot bajo el lenguaje de programación Java, siguiendo el patrón arquitectónico Modelo-Vista-Controlador (MVC), con el objetivo de garantizar la modularidad, mantenibilidad y separación de responsabilidades dentro del sistema.

Tecnologías principales:

- Spring Boot: framework principal utilizado para el desarrollo de la aplicación web y la gestión de dependencias del ecosistema Spring.
- Spring Data JPA: utilizado para la persistencia de datos y comunicación con la base de datos mediante entidades y repositorios.
- H2 Database: base de datos embebida utilizada durante el desarrollo y las pruebas del sistema.
- Thymeleaf: motor de plantillas empleado para la generación dinámica de vistas HTML.
- Maven: herramienta de gestión de dependencias, compilación y automatización del proyecto.
- JUnit 5: framework utilizado para la implementación de pruebas unitarias.
- Mockito: librería empleada para la simulación de dependencias durante las pruebas unitarias.
- JaCoCo: herramienta utilizada para medir la cobertura de código de los tests realizados.
- SonarQube / SonarCloud: plataforma utilizada para el análisis estático del código y control de calidad del proyecto.
- Git y GitHub: utilizados para el control de versiones y trabajo colaborativo mediante ramas.

3.2. COMPONENTES PRINCIPALES

- Entidades del sistema: Usuario, Inmueble, Reserva, Deseo y Propietario.
- Servicios: gestión de usuarios, reservas, autenticación y lógica de negocio del sistema.

PROYECTO ISO II

- Persistencia: acceso a datos mediante DAOs y entidades JPA.
- Controladores: coordinación entre las vistas y la lógica de negocio mediante controladores Spring MVC.
- Vistas: páginas HTML dinámicas desarrolladas con Thymeleaf para login, registro, gestión de inmuebles y reservas.
- Testing: implementación de pruebas unitarias para validar funcionalidades críticas del sistema y mejorar la calidad del software.

4. GESTIÓN DE CONFIGURACIÓN

4.1. CONTROL DE VERSIONES

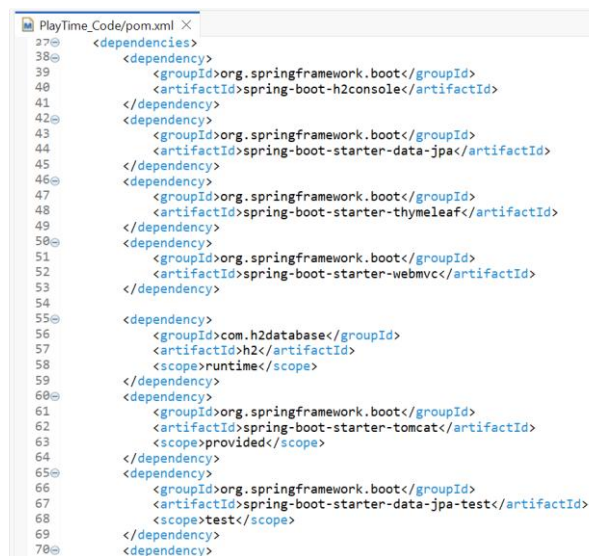
El proyecto se gestiona mediante Git y GitHub como sistema de control de versiones.

- main: rama estable para entregas
- develop: integración de funcionalidades en curso
- feature/*: ramas específicas por historia de usuario o tarea
- testing
- mantenimiento

Cada funcionalidad se desarrolló en ramas independientes antes de integrarse en develop.

4.2. GESTIÓN DE DEPENDENCIAS Y COMPILACIÓN

El archivo pom.xml contiene las dependencias necesarias del proyecto:



```
27<dependencies>
38<dependency>
39<groupId>org.springframework.boot</groupId>
40<artifactId>spring-boot-starter</artifactId>
41</dependency>
42<dependency>
43<groupId>org.springframework.boot</groupId>
44<artifactId>spring-boot-starter-data-jpa</artifactId>
45</dependency>
46<dependency>
47<groupId>org.springframework.boot</groupId>
48<artifactId>spring-boot-starter-thymeleaf</artifactId>
49</dependency>
50<dependency>
51<groupId>org.springframework.boot</groupId>
52<artifactId>spring-boot-starter-web</artifactId>
53</dependency>
54<dependency>
55<groupId>com.h2database</groupId>
56<artifactId>h2</artifactId>
57<scope>runtime</scope>
58</dependency>
59<dependency>
60<groupId>org.springframework.boot</groupId>
61<artifactId>spring-boot-starter-tomcat</artifactId>
62<scope>provided</scope>
63</dependency>
64<dependency>
65<groupId>org.springframework.boot</groupId>
66<artifactId>spring-boot-starter-data-jpa</artifactId>
67<scope>test</scope>
68</dependency>
69</dependencies>
```

4.3. INTEGRACIÓN Y DESPLIEGUE

- Servidor de aplicaciones: utilización de Tomcat embebido proporcionado por Spring Boot para la ejecución de la aplicación web.
- Base de datos: H2 Database embebida, integrada mediante Spring Data JPA para facilitar el desarrollo y las pruebas del sistema.
- Integración continua: uso de Git y GitHub para el control de versiones y trabajo colaborativo mediante ramas.

PROYECTO ISO II

- Pruebas realizadas: validación de funcionalidades críticas como inicio de sesión, registro de usuarios, persistencia de entidades y gestión de reservas.
- Control de calidad: integración de pruebas unitarias mediante JUnit y análisis de calidad del código utilizando SonarQube y JaCoCo.

Se plantea como mejora futura la automatización completa de pruebas y el despliegue del sistema en un entorno remoto de producción.

5. SONAR (ANÁLISIS INICIAL)

Durante esta entrega se ha realizado un análisis estático del código del proyecto utilizando la plataforma SonarQube, con el objetivo de evaluar la calidad, mantenibilidad y fiabilidad del software desarrollado hasta el momento.

5.1. RESULTADOS DEL ANÁLISIS INICIAL

Los resultados obtenidos en SonarQube antes de implementar los tests unitarios fueron los siguientes:

- Security: A — 0 issues
- Reliability: E — 19 issues
- Maintainability: A — 59 issues
- Security Hotspots Reviewed: E — 0.0%
- Coverage: 0.0%
- Duplications: 9.4%
- Quality Gate: FAILED

El estado del Quality Gate aparece como FAILED, principalmente porque el proyecto no tenía cobertura de pruebas unitarias en ese momento, mostrando un 0.0% de coverage. Además, existían Security Hotspots sin revisar y un número elevado de issues relacionados con fiabilidad y mantenibilidad.

A pesar de que la calificación de seguridad y mantenibilidad era buena, el análisis inicial reflejaba la necesidad de incorporar pruebas unitarias, revisar los problemas detectados y reducir la duplicación de código.

Estos resultados se tomaron como punto de partida para la fase de testing y mejora de calidad del proyecto.



5.2. RESULTADO SEGUNDO ANÁLISIS

Tras la implementación de pruebas unitarias mediante JUnit y la ejecución del análisis de calidad con SonarQube, se obtuvo una mejora significativa en los indicadores del proyecto.

PROYECTO ISO II

Los resultados obtenidos fueron los siguientes:

- Security: A — 0 issues
- Reliability: E — 19 issues
- Maintainability: A — 59 issues
- Security Hotspots Reviewed: E — 0.0%
- Coverage: 77.7%
- Duplications: 9.3%
- Quality Gate: FAILED

La mejora más destacable se produjo en la cobertura de código, que pasó de un 0.0% inicial a un 77.7%, gracias a la incorporación de pruebas unitarias.

Además, se consiguió una ligera reducción en la duplicación de código y una validación más completa de la lógica de negocio relacionada con usuarios, reservas y persistencia.

No obstante, el Quality Gate continuó apareciendo como FAILED debido principalmente a la existencia de Security Hotspots pendientes de revisión y a determinados problemas de fiabilidad detectados por SonarQube.

Estos resultados reflejan una mejora importante en la calidad y verificabilidad del sistema tras la fase de testing implementada.



5.3. PROPUESTA DE CONFIGURACIÓN ALTERNATIVA

En la versión de SonarQube/SonarCloud utilizada para el proyecto no se dispone de permisos de administración suficientes para modificar directamente la configuración global del análisis. No obstante, se plantea una propuesta de configuración alternativa orientada a mejorar la precisión de los resultados obtenidos.

La configuración propuesta consiste en excluir del análisis los archivos HTML correspondientes a las vistas Thymeleaf del proyecto.

Justificación:

- Los archivos HTML no contienen lógica de negocio Java relevante para el análisis principal del sistema.
- Parte de los code smells detectados en las vistas no afectan directamente a la mantenibilidad de la lógica backend.
- La exclusión de estas plantillas permitiría centrar el análisis en controladores, servicios, persistencia y entidades del proyecto.

PROYECTO ISO II

- Esta práctica es habitual en proyectos desarrollados con Spring Boot y Thymeleaf cuando el objetivo principal del análisis es evaluar la calidad del código Java.

Configuración propuesta: `src/main/resources/templates/**`

Pasos teóricos para aplicar la configuración:

1. Acceder a la administración del proyecto en SonarQube.
2. Entrar en General Settings → Analysis Scope.
3. Añadir la ruta anterior en el apartado Source File Exclusions.
4. Guardar la configuración.
5. Ejecutar nuevamente el análisis mediante Maven.

5.4. ISSUES SELECCIONADAS Y EXPLICACIÓN

A continuación, se describen los issues más relevantes detectados durante el análisis:

1. JAVA:S1186 — MÉTODOS VACÍOS SIN JUSTIFICACIÓN

Campo	Detalle
Tipo	Code Smell
Severidad	Critical
Regla	Methods should not be empty
Descripción	SonarQube detecta métodos o constructores con cuerpo vacío sin ninguna explicación, lo que dificulta saber si es un olvido o algo intencionado.
Solución aplicada	Añadir un comentario interior que justifique por qué el método está vacío. En entidades JPA, el constructor vacío es obligatorio para que el framework pueda instanciar la clase por reflexión.
Ejemplo de fix	<pre>public Inmuble() { // Requerido por JPA }</pre>
Commit tipo	<code>fix(S1186):</code> añadir comentario justificativo en constructor vacío de <Clase>

2. JAVA:S2699 — TESTS SIN ASSERTIONS

Campo	Detalle
Tipo	Code Smell
Severidad	Blocker
Regla	Tests should include assertions

PROYECTO ISO II

Descripción	Un test que no contiene ninguna assertion no verifica nada de forma explícita, por lo que puede pasar aunque el sistema falle.
Solución aplicada	Injectar el ApplicationContext y añadir <code>assertThat(applicationContext).isNotNull()</code> para verificar que el contexto de Spring arranca correctamente.
Ejemplo de fix	<code>assertThat(applicationContext).isNotNull();</code>
Commit tipo	<code>fix(S2699): añadir assertion al test</code> <code><nombreTest> en <Clase></code>

3. JAVA:S3753 — @SESSIONATTRIBUTES SIN LLAMADA A SETCOMPLETE()

Campo	Detalle
Tipo	Bug
Severidad	Blocker
Regla	<code>@Controller</code> classes that use <code>@SessionAttributes</code> must call <code>setComplete</code> on their <code>SessionStatus</code> objects
Descripción	Cuando un <code>@Controller</code> usa <code>@SessionAttributes</code> , Spring almacena atributos en sesión hasta que se invoca <code>sessionStatus.setComplete()</code> . Si nunca se llama, los atributos persisten indefinidamente en la sesión del usuario, pudiendo causar comportamientos inesperados o fugas de datos entre peticiones.
Solución aplicada	Añadir un parámetro <code>SessionStatus</code> en el método <code>@RequestMapping</code> correspondiente e invocar <code>sessionStatus.setComplete()</code> al finalizar el flujo (p.ej. en <code>logout</code> o al completar la operación).
Ejemplo de fix	<pre>public String logout(SessionStatus sessionStatus) { sessionStatus.setComplete(); return "redirect:/login"; }</pre>
Commit tipo	<code>fix(S3753): añadir setComplete() en</code> <code>SessionStatus</code> de <code><Clase></code>

PROYECTO ISO II

4. JAVA:S3776 — COMPLEJIDAD COGNITIVA EXCESIVA

Campo	Detalle
Tipo	Code Smell
Severidad	Critical
Regla	Cognitive Complexity of methods should not be too high
Descripción	Un método con demasiados if, bucles y condiciones anidadas es difícil de leer, testear y mantener. SonarQube permite un máximo de 15 puntos de complejidad cognitiva por método.
Solución aplicada	Extraer cada bloque lógico independiente a métodos privados auxiliares, de forma que el método principal orqueste el flujo sin acumular complejidad.
Ejemplo de fix	Extraer cada filtro a filtrarPorCiudad(), filtrarPorSearch(), etc., y delegar desde un método aplicarFiltros().
Commit tipo	fix(S3776): reducir complejidad cognitiva de <método> extrayendo lógica a métodos privados

5. JAVA:S1192 — LITERALES DE CADENA DUPLICADOS

Campo	Detalle
Tipo	Code Smell
Severidad	Critical
Regla	String literals should not be duplicated
Descripción	Repetir el mismo literal de cadena 3 o más veces dificulta el mantenimiento: si hay que cambiarlo, hay que buscarlo en todos los sitios donde aparece, con riesgo de olvidar alguno.
Solución aplicada	Extraer el literal repetido a una constante <code>private static final String</code> en la misma clase y reemplazar todas las ocurrencias por la constante.
Ejemplo de fix	<code>private static final String ATTR_ERROR = "error";</code> y usar <code>model.addAttribute(ATTR_ERROR, "mensaje")</code> en todos los métodos.

PROYECTO ISO II

Commit tipo

fix(S1192): extraer literal
'<valor>' a constante en <Clase>

5.5. RESULTADO ULTIMO ANÁLISIS

Tras la implementación de pruebas unitarias mediante JUnit y la ejecución del último análisis de calidad con SonarQube, se observó una mejora en diferentes indicadores del proyecto.

Los resultados obtenidos fueron los siguientes:

- Security: A — 0 issues
- Reliability: E — 12 issues
- Maintainability: A — 46 issues
- Security Hotspots Reviewed: E — 0.0%
- Coverage: 78.6%
- Duplications: 9.2%
- Quality Gate: FAILED

La mejora más destacable se produjo en la cobertura de código, alcanzando un 78.6% gracias a la incorporación y ampliación de pruebas unitarias sobre controladores, servicios y entidades principales del sistema.

Además, se consiguió mantener un nivel alto de mantenibilidad del código, obteniendo calificación A en SonarQube, así como una ausencia de problemas de seguridad detectados en el análisis.

No obstante, el Quality Gate continuó apareciendo como FAILED debido principalmente a la existencia de problemas de fiabilidad detectados por SonarQube y a la presencia de Security Hotspots pendientes de revisión.

También se detectó un porcentaje de duplicación de código del 9.2%, derivado principalmente de estructuras repetidas en determinados controladores y pruebas unitarias.

Estos resultados reflejan una mejora importante en la calidad, mantenibilidad y verificabilidad del sistema tras la fase de testing y mantenimiento implementada.



5.6. CONCLUSIONES DEL ANÁLISIS

Los análisis realizados mediante SonarQube a lo largo del desarrollo del proyecto permitieron evaluar de forma continua la calidad del código y detectar distintos problemas relacionados con cobertura, mantenibilidad, fiabilidad y duplicación de código.

PROYECTO ISO II

En el análisis inicial se observó una ausencia total de pruebas unitarias, reflejada en un 0.0% de cobertura, así como un número elevado de incidencias relacionadas con fiabilidad y mantenibilidad. Además, el estado del Quality Gate aparecía como FAILED debido principalmente a la falta de testing y a la existencia de Security Hotspots pendientes de revisión.

Tras la implementación progresiva de pruebas unitarias con JUnit y Mockito, junto con la incorporación de JaCoCo para medir cobertura, se consiguió una mejora significativa en los indicadores del proyecto. La cobertura alcanzó finalmente un 78.6%, permitiendo validar gran parte de la lógica de negocio relacionada con usuarios, inmuebles, reservas y controladores principales del sistema.

Además, durante la fase de mantenimiento se corrigieron distintos code smells y problemas detectados por SonarQube, realizando tareas de refactorización y reorganización del código. Como resultado, se consiguió reducir las incidencias de mantenibilidad de 59 a 46 y los problemas de fiabilidad de 19 a 12.

El análisis también permitió identificar aspectos susceptibles de mejora futura, especialmente relacionados con la revisión de Security Hotspots y la reducción adicional de duplicación de código en determinados controladores y tests.

En conjunto, el uso de SonarQube, JaCoCo y pruebas unitarias permitió mejorar considerablemente la calidad, estabilidad y verificabilidad del sistema, facilitando además el mantenimiento y evolución futura del proyecto.

5.7. PRUEBAS UNITARIAS (JUNIT)

5.7.1. INTRODUCCIÓN A LAS PRUEBAS UNITARIAS

Para mejorar la fiabilidad del proyecto y aumentar la cobertura se añadieron pruebas unitarias con JUnit 5, cubriendo principalmente:

- Servicios/gestores (lógica de negocio)
- Entidades del dominio (getters/setters y comportamiento básico)
- Controladores con pruebas de tipo MVC

La ejecución de pruebas se realiza con Maven:

```
- mvn test
- mvn clean test
```

5.7.2. TESTS IMPLEMENTADOS (7 EXPLICADOS)

Para la validación del sistema se han definido casos de prueba basados en clases de equivalencia y comprobación de condiciones límite, siguiendo un enfoque sistemático de testing. A continuación, se describen los principales casos de prueba implementados.

PROYECTO ISO II

5.7.2.1. POLITICACANCELACIONTEST

A) Valores del enum

Caso	Campo probado	Resultado esperado
CP1	REEMBOLSABLE	Nombre correcto
CP2	NO_REEMBOLSABLE	Nombre correcto
CP3	REEMBOLSABLE_50	Nombre correcto

B) Método valueOf()

Caso	Campo probado	Resultado esperado
CP4	"REEMBOLSABLE"	Devuelve PoliticaCancelacion.REEMBOLSABLE

Este test verifica el correcto funcionamiento del enum PoliticaCancelacion, utilizado para representar las distintas políticas de cancelación disponibles en los inmuebles del sistema.

Las pruebas realizadas validan que los diferentes valores definidos dentro del enum están correctamente implementados y accesibles desde la aplicación.

En concreto, se comprueba:

- La correcta identificación de los estados:
 - REEMBOLSABLE
 - NO_REEMBOLSABLE
 - REEMBOLSABLE_50
- El correcto funcionamiento del método valueOf(), utilizado para convertir cadenas de texto en valores del enum.

Aunque se trata de un test sencillo, resulta importante para garantizar la estabilidad de las políticas de cancelación utilizadas dentro de la lógica de reservas y cancelaciones del sistema.

5.7.2.2. RESERVASERVICETEST

A) Método: crearReserva (Long inquilinoId, Long inmuebleId, LocalDate inicio, LocalDate fin)

Caso	Inquilino ID	Inmueble ID	Fechas	Resultado esperado	Verificación adicional
CP1	1	2	Válidas	Reserva creada correctamente	Se llama al DAO una vez
CP2	null	2	Válidas	Excepción IllegalArgumentException	No se llama al DAO
CP3	1	2	Fechas ocupadas	Excepción IllegalStateException	No se crea reserva

B) Método: crearSolicitudReserva(Long inquilinoId, Long inmuebleId, LocalDate inicio, LocalDate fin)

Caso	Datos válidos	Estado inicial	Resultado esperado
CP4	Sí	PENDIENTE	Reserva creada correctamente

PROYECTO ISO II

C) Método: listarTodas()

Caso	Resultado del repositorio	Resultado esperado
CP5	Lista con 2 reservas	Tamaño de lista = 2

D) Método: listarPorInquilino(Long id)

Caso	Reservas encontradas	Resultado esperado
CP6	1 reserva	Tamaño de lista = 1

E) Método: confirmar(Long idReserva)

Caso	Reserva existente	Resultado esperado	Verificación adicional
CP7	Sí	Estado = CONFIRMADO	Se actualiza en DAO

F) Método: rechazar(Long idReserva)

Caso	Política cancelación	Resultado esperado	Verificación adicional
CP8	REEMBOLSABLE	Estado = RECHAZADA	Se actualiza en DAO

G) Método: cancelar(Long idReserva)

Caso	Política cancelación	Resultado esperado	Verificación adicional
CP9	REEMBOLSABLE	Estado = CANCELADA	Se actualiza en DAO

H) Método: obtenerFechasOcupadas(Long inmuebleId)

Caso	Reservas confirmadas	Resultado esperado
CP10	Reserva del 1 al 3 de mayo	Lista con 3 fechas ocupadas

El test verifica la lógica de negocio de la clase ReservaService, asegurando el correcto funcionamiento de las operaciones relacionadas con la creación, gestión y validación de reservas.

Para ello, se simula el comportamiento del repositorio ReservaDAO mediante Mockito, evitando el acceso real a la base de datos y permitiendo realizar pruebas unitarias aisladas.

El test verifica los siguientes aspectos:

- Creación correcta de reservas cuando los datos son válidos.
- Validación de parámetros obligatorios.
- Detección de solapamiento de fechas ocupadas.
- Creación de solicitudes de reserva pendientes.
- Listado correcto de reservas del sistema.
- Confirmación, rechazo y cancelación de reservas.
- Obtención de fechas ocupadas para un inmueble concreto.
- Verificación de llamadas al repositorio mediante Mockito.

Además, se comprueba que los métodos del DAO únicamente se ejecutan cuando las validaciones previas son correctas, evitando operaciones innecesarias sobre persistencia.

PROYECTO ISO II

Este conjunto de pruebas contribuye a mejorar la fiabilidad de la lógica de reservas y garantiza un comportamiento estable del sistema ante diferentes escenarios de uso.

5.7.2.3. USUARIOTEST

A) Constructor

Caso	Acción	Resultado esperado
CP1	Crear nuevo usuario vacío	Crear nuevo usuario vacío
CP2	Crear nuevo usuario vacío	Lista de deseos inicializada
CP3	Crear nuevo usuario vacío	Lista de deseos vacía

B) Getters y Setters

Caso	Campo probado	Resultado esperado
CP4	id	Valor almacenado correctamente
CP5	login	Valor almacenado correctamente
CP6	pass	Valor almacenado correctamente
CP7	nombre	Valor almacenado correctamente
CP8	apellidos	Valor

El test realizado sobre la entidad `Usuario` tiene como objetivo verificar el correcto funcionamiento de los atributos y métodos básicos de la clase, asegurando la consistencia de la información almacenada en los usuarios del sistema.

Al tratarse de una entidad JPA, el test se centra en validar:

- La correcta creación de objetos mediante el constructor vacío.
- La inicialización adecuada de la lista de deseos del usuario.
- El correcto funcionamiento de los métodos getters y setters.
- La validación de los métodos auxiliares relacionados con los roles del usuario.
- La correcta gestión de la lista de deseos del usuario mediante los métodos `getListaDeseos()` y `setListaDeseos()`.

El test comprueba que los atributos del usuario almacenan y recuperan correctamente valores como identificador, login, contraseña, nombre, dirección y rol asignado.

Además, se validan los métodos:

- ``esPropietario()``
- ``esInquilino()``
- ``esUsuarioNormal()``

Asegurando que devuelven el resultado esperado según el rol configurado para cada usuario.

Por otro lado, también se verifica el correcto funcionamiento de la lista de deseos, comprobando que es posible almacenar inmuebles asociados al usuario y recuperarlos correctamente.

PROYECTO ISO II

Estas pruebas permiten garantizar la estabilidad de la entidad `Usuario`, evitando errores de inicialización y asegurando el correcto comportamiento de una de las entidades principales del sistema.

5.7.2.4. INMUEBLETEST

A) Constructor

Caso	Acción	Resultado esperado
CP1	Crear nuevo inmueble vacío	Objeto creado correctamente
CP2	Crear nuevo inmueble vacío	Política de cancelación inicializada

B) Getters y Setters

Caso	Campo probado	Resultado esperado
CP3	id	Valor almacenado correctamente
CP4	titulo	Valor almacenado correctamente
CP5	descripción	Valor almacenado correctamente
CP6	dirección	Valor almacenado correctamente
CP7	ciudad	Valor almacenado correctamente
CP8	precio por noche	Valor almacenado correctamente
CP9	número de habitaciones	Valor almacenado correctamente
CP10	número de baños	Valor almacenado correctamente
CP11	reserva inmediata	Valor almacenado correctamente
CP12	política de cancelación	Valor almacenado correctamente
CP13	propietario	Usuario propietario asociado correctamente

C) Política de cancelación por defecto

Caso	Acción	Resultado esperado
CP14	Crear inmueble sin modificar política	Política por defecto = REEMBOLSABLE

D) Reserva inmediata

Caso	Acción	Resultado esperado
CP15	Activar reserva inmediata	isReservaInmediata() devuelve true
CP16	Desactivar reserva inmediata	isReservaInmediata() devuelve false

Este test verifica el correcto funcionamiento de la entidad Inmueble, una de las clases principales del dominio del proyecto, ya que representa las propiedades que pueden ser publicadas y reservadas por los usuarios.

Al tratarse de una entidad JPA, las pruebas se centran en comprobar la correcta inicialización del objeto, el funcionamiento de sus métodos de acceso y la consistencia de los valores almacenados.

El test verifica los siguientes aspectos:

PROYECTO ISO II

- Creación correcta de un inmueble mediante el constructor vacío.
- Inicialización de la política de cancelación por defecto.
- Correcto funcionamiento de los métodos getters y setters.
- Almacenamiento de datos principales del inmueble, como título, descripción, dirección, ciudad, precio, habitaciones y baños.
- Gestión del valor booleano de reserva inmediata.
- Asignación correcta de la política de cancelación.
- Asociación del inmueble con un usuario propietario.

Además, se comprueba que la política de cancelación por defecto es REEMBOLSABLE, lo que garantiza que el inmueble mantiene un estado inicial válido, aunque no se configure manualmente.

Este test permite asegurar que la entidad Inmueble mantiene correctamente sus datos y relaciones, evitando errores de inicialización y contribuyendo a la estabilidad de la lógica relacionada con la gestión de propiedades.

5.7.2.5. RESERVATEST

A) Constructor vacío

Caso	Acción	Resultado esperado
CP1	Crear nueva reserva vacía	Objeto creado correctamente

B) Constructor completo

Caso	Datos proporcionados	Resultado esperado
CP2	Usuario, inmueble, fechas, precio y estado válidos	Reserva creada correctamente con todos los atributos asignados

C) Getters y Setters

Caso	Campo probado	Resultado esperado
CP3	id	Valor almacenado correctamente
CP4	inquilino	Usuario asociado correctamente
CP5	inmueble	Inmueble asociado correctamente
CP6	fecha inicio	Valor almacenado correctamente
CP7	fecha fin	Valor almacenado correctamente
CP8	precio total	Valor almacenado correctamente
CP9	estado	Valor almacenado correctamente

D) Enum EstadoReserva

Caso	Campo probado	Resultado esperado
CP10	CONFIRMADA	Nombre del estado correcto
CP11	RECHAZADA	Nombre del estado correcto
CP12	CANCELADA	Nombre del estado correcto

PROYECTO ISO II

E) Método toString()

Caso	Datos de la reserva	Resultado esperado
CP13	id, precio y estado definidos	Texto contiene los valores esperados

El test realizado sobre la entidad Reserva tiene como objetivo verificar el correcto funcionamiento de los atributos, constructores y métodos auxiliares relacionados con la gestión de reservas del sistema.

La entidad Reserva representa una de las piezas centrales de la aplicación, ya que conecta usuarios, inmuebles y fechas de reserva dentro de la lógica de negocio.

Las pruebas implementadas verifican:

- La correcta creación de objetos mediante constructor vacío.
- El funcionamiento del constructor completo con todos los parámetros.
- El correcto almacenamiento y recuperación de atributos mediante getters y setters.
- La correcta asociación entre reservas, usuarios e inmuebles.
- El funcionamiento del enum EstadoReserva.
- La correcta representación textual del objeto mediante el método toString().

Además, se valida que los diferentes estados posibles de una reserva (CONFIRMADA, RECHAZADA y CANCELADA) se gestionan correctamente dentro de la entidad.

Estas pruebas permiten asegurar la estabilidad de la entidad Reserva, garantizando la integridad de la información relacionada con las reservas y evitando errores en la gestión de estados y datos asociados.

5.7.2.6. MENUSOLICITUDESCONTROLLERTEST

A) Método: verSolicitudes(Usuario usuario, Model model, SessionStatus status)

Caso	Usuario recibido	Reservas existentes	Resultado esperado
CP1	null	No aplica	Redirección a login
CP2	Propietario válido	Reservas propias y ajenas	Devuelve la vista menu_solicitudes_reserva
CP3	Propietario válido	Reservas pendientes	Se añaden las solicitudes filtradas al modelo

B) Método: confirmarReserva(Long id)

Caso	ID reserva	Resultado esperado	Verificación adicional
CP4	5	Redirección a solicitudes	Se llama a reservaService.confirmar(5L)

C) Método: rechazarReserva(Long id)

Caso	ID reserva	Resultado esperado	Verificación adicional
CP5	5	Redirección a solicitudes	Se llama a reservaService.rechazar(5L)

PROYECTO ISO II

Este test valida el comportamiento del controlador MenuSolicitudesController, encargado de gestionar las solicitudes de reserva desde la perspectiva del propietario.

A diferencia de los tests de entidades o servicios, este test se centra en comprobar la lógica de navegación y la interacción entre el controlador, los servicios y el modelo de la vista.

Para ello, se utilizan mocks de InmuebleService, ReservaService y Model, evitando depender de la capa de persistencia o de una ejecución real de la aplicación web.

El test comprueba los siguientes aspectos:

- Si no existe usuario autenticado, el controlador redirige correctamente a la página de login.
- Si existe un propietario válido, el controlador obtiene los inmuebles y reservas asociados, filtra las solicitudes correspondientes y devuelve la vista adecuada.
- El modelo recibe el atributo solicitudes, necesario para mostrar las reservas pendientes en la vista.
- Al confirmar una reserva, se invoca el servicio correspondiente y se redirige de nuevo al menú de solicitudes.
- Al rechazar una reserva, se invoca el método adecuado del servicio y se devuelve la redirección esperada.

Tras las modificaciones realizadas en el controlador, se añadió el parámetro SessionStatus para adaptarse al flujo actualizado de sesiones de Spring MVC.

Este test resulta importante porque permite validar el correcto funcionamiento de la capa de controladores, asegurando que las peticiones del usuario se gestionan correctamente y que la aplicación redirige a las vistas adecuadas según cada situación.

5.7.2.7. INMUEBLESERVICETEST

A) Método: registrarInmueble(...)

Caso	Datos del inmueble	Comportamiento del DAO	Resultado esperado
CP1	Datos completos válidos	Guarda correctamente	Devuelve true
CP2	Datos completos válidos	Lanza excepción	Devuelve false
CP3	Versión corta con datos válidos	Guarda correctamente	Devuelve true

B) Método eliminar(Long id)

Caso	Inmueble encontrado	Resultado esperado	Verificación adicional
CP4	Sí	Se elimina el inmueble	Se llama a eliminar()
CP5	No	No se elimina nada	No se llama a eliminar()

PROYECTO ISO II

C) Método: listarTodos()

Caso	Resultado del DAO	Resultado esperado
CP6	Lista con 2 inmuebles	Lista tamaño 2

D) Método: buscarInmuebles(String texto)

Caso	Texto de búsqueda	Resultado del DAO	Resultado esperado
CP7	"piso"	Lista con 1 inmueble	Lista tamaño 1

E) Método: buscarPorId(Long id)

Caso	ID buscado	Resultado del DAO	Resultado esperado
CP8	1	Inmueble existente	Devuelve un inmueble no nulo

Este test valida la lógica de negocio de la clase InmuebleService, encargada de gestionar las operaciones relacionadas con los inmuebles del sistema.

Para ello, se utiliza Mockito para simular el comportamiento de InmuebleDAO, evitando depender de la base de datos real y permitiendo realizar pruebas unitarias aisladas sobre la capa de servicio.

Las pruebas implementadas verifican los siguientes aspectos:

- Registro correcto de un inmueble con datos válidos.
- Control de errores cuando se produce una excepción durante el guardado.
- Registro mediante la versión corta del método registrarInmueble.
- Eliminación de inmuebles existentes.
- Comportamiento correcto cuando se intenta eliminar un inmueble inexistente.
- Listado completo de inmuebles.
- Búsqueda de inmuebles por texto.
- Búsqueda de inmuebles por identificador.

Además, se comprueba mediante verify() que los métodos del DAO se invocan únicamente cuando corresponde. Por ejemplo, si el inmueble no existe, no se llama al método eliminar().

Este test resulta importante porque valida una de las funcionalidades principales de la aplicación: la gestión de inmuebles publicados por los propietarios. Al cubrir tanto casos correctos como situaciones de error, contribuye a mejorar la fiabilidad del sistema y facilita el mantenimiento futuro del código.

5.8. INFORMES DE PRUEBAS Y COBERTURA (SUREFIRE / JACOCO)

Además de la ejecución de pruebas unitarias, se generaron informes automáticos para analizar el estado del testing y la cobertura del código del proyecto.

Las herramientas utilizadas fueron las siguientes:

- Maven Surefire Plugin: encargado de ejecutar automáticamente los tests JUnit y generar informes sobre el resultado de las pruebas.

PROYECTO ISO II

- JaCoCo: herramienta utilizada para medir la cobertura de código alcanzada por las pruebas unitarias implementadas.

Comandos utilizados:

```
- mvn clean test
- mvn site
- mvnw.cmd clean verify site:site
```

Los informes generados permitieron comprobar el aumento de cobertura conseguido tras la implementación de pruebas unitarias, especialmente en la capa de servicios y entidades del sistema.

Las clases relacionadas con la lógica de negocio (business.service) alcanzaron niveles elevados de cobertura, permitiendo validar operaciones críticas como:

- Registro e inicio de sesión de usuarios
- Gestión y validación de reservas
- Persistencia de entidades
- Comprobación de roles y listas de deseos

Asimismo, los informes de JaCoCo permitieron identificar métodos sin cubrir completamente y posibles mejoras futuras en el testing del proyecto.

Estos resultados fueron posteriormente reflejados en SonarQube para evaluar la evolución de la calidad y cobertura del sistema.

Elemento	Instrucciones omitidas	Cov.	Ramas perdidas	Cov.	Omitido	Cxty	Omitido	Pauta	Omitido	Métodos	Omitido	Clases	
es.ucim.PlayTime_Code.persistence		0%		0%	28	28	71	71	19	19	3	3	
es.ucim.PlayTime_Code.business.service		87%		60%	27	55	10	116	0	20	0	3	
es.ucim.PlayTime_Code.business.controller		97%		74%	47	139	6	270	0	48	0	7	
es.ucim.PlayTime_Code		0%		n / A	4	4	5	5	4	4	2	2	
es.ucim.PlayTime_Code.business.entity		100%		100%	0	75	0	108	0	72	0	7	
Total		420 de 2378	82%	92 de 273	66%	106	301	92	570	23	163	5	22

5.9. MANTENIMIENTO DEL SOFTWARE

Para la realización del mantenimiento del sistema se ha definido un plan de mantenimiento orientado a mejorar la calidad del código, reducir errores y aumentar la cobertura de pruebas del proyecto PlaytimeCode.

El mantenimiento se ha llevado a cabo utilizando una rama específica denominada mantenimiento, siguiendo una estrategia de desarrollo basada en ramas para evitar afectar al funcionamiento estable del sistema durante la implementación de mejoras y correcciones.

Durante esta fase se han realizado distintos tipos de mantenimiento:

5.9.1. MANTENIMIENTO CORRECTIVO

Se han corregido distintos errores detectados durante el desarrollo y mediante el análisis de SonarQube. Entre las correcciones realizadas destacan:

- Corrección de conflictos y errores en controladores.
- Revisión de métodos de servicios relacionados con reservas.

PROYECTO ISO II

- Solución de incidencias derivadas de merges entre ramas del proyecto.

5.9.2. MANTENIMIENTO PERFECTIVO

Se han realizado mejoras orientadas a aumentar la calidad y mantenibilidad del código:

- Refactorización de diferentes clases del proyecto.
- Mejora de la organización del código.
- Optimización de controladores y servicios.

5.9.3. MANTENIMIENTO PREVENTIVO

Con el objetivo de prevenir futuros errores y facilitar la evolución del sistema:

- Se ampliaron los tests JUnit del proyecto.
- Se incrementó la cobertura de pruebas.
- Se realizaron análisis continuos mediante SonarQube.
- Se revisaron métricas de calidad, bugs, vulnerabilidades y code smells.

Una vez implementadas las tareas de mantenimiento, se ejecutaron nuevamente los análisis de SonarQube para comprobar la mejora de calidad del sistema. Tras las modificaciones realizadas se observó una reducción de incidencias detectadas y una mejora en los indicadores de mantenibilidad y cobertura de código.

Finalmente, los cambios desarrollados en la rama mantenimiento fueron integrados en la rama develop mediante merge, permitiendo consolidar las mejoras realizadas dentro del flujo principal de desarrollo del proyecto.

5.10. COMPARATIVA DE MÉTRICAS

Métrica	Análisis inicial	Análisis final
Maintainability	59	46
Reliability	19	12
Coverage	0.0%	78.6%
Duplications	9.4%	9.2%
Quality Gate	FAILED	FAILED

Se observa una mejora significativa en la cobertura de código y una reducción de incidencias de mantenibilidad y fiabilidad tras la fase de testing y mantenimiento del proyecto.

6. CONCLUSIÓN

El desarrollo del proyecto PlaytimeCode ha permitido aplicar de forma práctica los conocimientos adquiridos en la asignatura de Ingeniería del Software II, utilizando herramientas, metodologías y tecnologías propias de un entorno profesional de desarrollo software.

A lo largo del proyecto se ha seguido una metodología ágil basada en Scrum, organizando el trabajo mediante sprints, backlog y control de versiones con Git y GitHub. Esta organización ha permitido dividir el desarrollo en fases progresivas, facilitando la implementación de funcionalidades, la integración del trabajo colaborativo y el seguimiento continuo del estado del sistema.

PROYECTO ISO II

Desde el punto de vista técnico, se desarrolló una aplicación web basada en Spring Boot y arquitectura MVC, incorporando funcionalidades relacionadas con autenticación de usuarios, gestión de inmuebles, reservas, listas de deseos, filtros de búsqueda y políticas de cancelación. Además, se integraron tecnologías complementarias como JPA, Thymeleaf, Maven y H2 Database, permitiendo construir un sistema funcional y estructurado.

Uno de los aspectos más importantes del proyecto ha sido la incorporación de testing y herramientas de calidad software. La implementación de pruebas unitarias mediante JUnit y Mockito permitió validar el comportamiento de entidades, servicios y controladores principales del sistema. Gracias a ello, la cobertura del código pasó de un 0.0% inicial a un 78.6% en el análisis final realizado con JaCoCo y SonarQube.

Asimismo, el uso de SonarQube permitió detectar problemas de mantenibilidad, fiabilidad y duplicación de código, facilitando la realización de tareas de mantenimiento correctivo, preventivo y perfectivo sobre el proyecto. Como resultado, se consiguió reducir significativamente el número de incidencias detectadas y mejorar la calidad general del software.

Aunque el Quality Gate final continuó apareciendo como FAILED debido principalmente a determinados problemas de fiabilidad y Security Hotspots pendientes de revisión, el proyecto mostró una evolución muy positiva en términos de calidad, cobertura y mantenibilidad respecto al análisis inicial.

En conclusión, el proyecto ha permitido desarrollar una aplicación funcional siguiendo buenas prácticas de ingeniería del software, aplicando metodologías ágiles, testing automatizado, control de calidad y trabajo colaborativo. Además, ha servido para adquirir experiencia práctica en el uso de herramientas profesionales de desarrollo y mantenimiento software, reforzando conocimientos fundamentales relacionados con calidad, testing y gestión de proyectos.